

Kitchen Management App — Architettura del Sistema

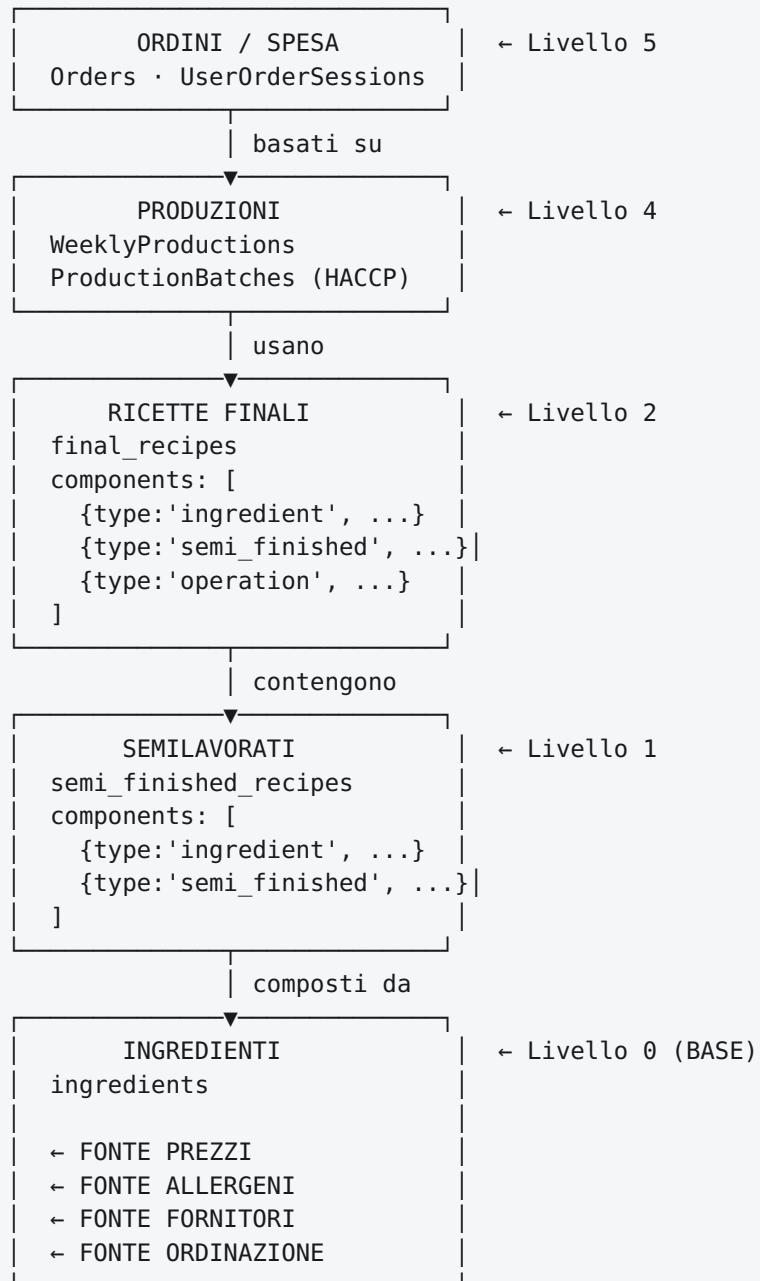
Documento generato il: 2026-03-16 **Stack:** React + Vite (frontend) · Express + tRPC (backend) · Drizzle ORM + MySQL (database)

1. Visione d'insieme (3 livelli)



2. Gerarchia dati — La piramide degli ingredienti

Gli **ingredienti** sono la BASE assoluta di ogni calcolo di costo, allergeni, produzione e ordinazione. Ogni dato a livello superiore dipende da loro.



3. Schema tabelle principali

3.1 ingredients — Parametri obbligatori e opzionali

Campo	Tipo	Obbligatorio	Descrizione
id	UUID (varchar)	☒ auto	Identificatore univoco
storeId	varchar	☒	Punto vendita di appartenenza
name	varchar(255)	☒	Nome ingrediente (univoco per store)
category	enum	☒	Additivi / Carni / Farine / ... (13 tipi)
unitType	enum u\ k	☒	u =unità, k =kg
packageQuantity	decimal(10,3)	☒	Quantità per confezione (es. 5 = 5kg)
packagePrice	decimal(10,2)	☒	Prezzo confezione (€)
pricePerKgOrUnit	decimal(10,2)	☒ auto-calc	= packagePrice / packageQuantity
supplierId	UUID	☐	FK → suppliers (fuzzy match all'import)
supplier	varchar	☐	Nome fornitore testuale
packageType	enum	☐	Sacco / Busta / Brick / ...
department	enum	☐ def:Cucina	Cucina Sala
minOrderQuantity	decimal	☐	Quantità minima ordine
packageSize	decimal	☐	Dimensione confezione
brand	varchar	☐	Marca
notes	text	☐	Note libere
isFood	boolean	☐ def:true	È un alimento (vs packaging)
isActive	boolean	☐ def:true	Attivo/disattivato
isOrderable	boolean	☐ def:true	Ordinabile
isSellable	boolean	☐ def:true	Vendibile
isSalaItem	boolean	☐ def:false	Articolo per la sala
isSoldByPackage	boolean	☐ def:false	Ordinato a confezione intera
subcategory	varchar	☐	Sottocategoria libera
allergens	json (array)	☐ def:[]	Lista allergeni EU (es. ["Glutine"])

3.2 semi_finished_recipes

Campo	Tipo	Obbligatorio	Descrizione
id	UUID	☒ auto	
storeId	varchar	☒	
code	varchar(50)	☒	Codice univoco per store
name	varchar	☒	
category	enum	☒	SPEZIE / SALSE / VERDURA / CARNE / ALTRO
finalPricePerKg	decimal	☒	Costo finale per kg
yieldPercentage	decimal	☒	Resa % (output / input × 100)
shelfLifeDays	int	☒	Durata in giorni
storageMethod	text	☒	Metodo conservazione
components	json	☐	Array di componenti
totalQuantityProduced	decimal	☐	Quantità totale prodotta
productionSteps	json	☐	Passi di produzione

3.3 final_recipes

Campo	Tipo	Obbligatorio	Descrizione
id	UUID	☒ auto	
storeId	varchar	☒	
code	varchar(50)	☒	Codice univoco per store
name	varchar	☒	
category	enum	☒	Pane / Carne / Salse / Verdure / ...
yieldPercentage	decimal	☒	Resa produzione %
conservationMethod	text	☒	Metodo conservazione
maxConservationTime	varchar	☒	Tempo max conservazione (es. "48 ore")
totalCost	decimal	☒ auto-calc	Somma costi componenti
components	json	☒	Array componenti (vedi §4)
serviceWastePercentage	decimal	☐ auto-calc	Scarto al servizio %
unitWeight	decimal	☐	Peso finale (kg)
producedQuantity	decimal	☐	Pezzi prodotti
measurementType	enum	☐ def:weight	weight_only / unit_only / both
pieceWeight	decimal	☐	Peso singolo pezzo (kg)
isSemiFinished	boolean	☐ def:false	Usata anche come semilavorato
isSellable	boolean	☐ def:true	Vendibile nel menu
isActive	boolean	☐ def:true	Visibile / Nascosta
sellingPrice	decimal	☐	Prezzo di vendita

4. Struttura JSON componenti — CRITICO

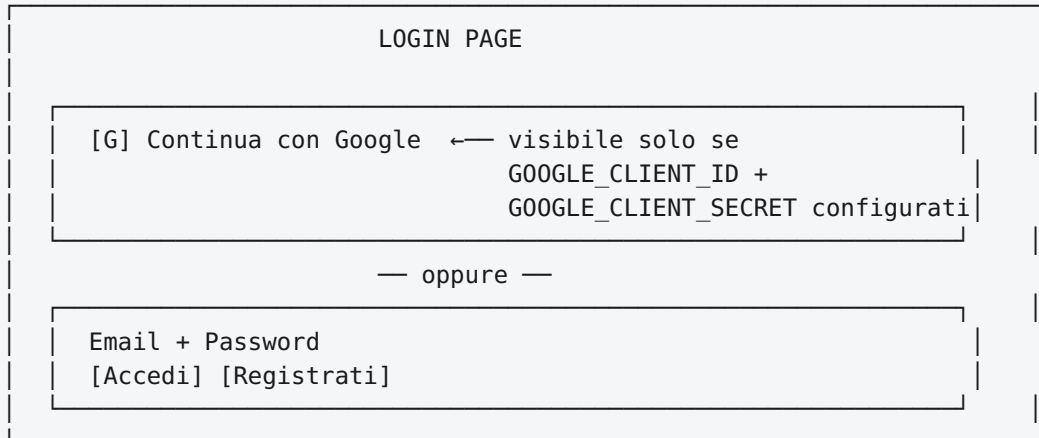
I componenti nelle ricette sono salvati come JSON array. Esiste una **doppia convenzione** (storica vs nuova) che causa bug se non normalizzata:

```
// ✅ Convenzione CORRETTA (usata da finalRecipes e semiFinishedRecipes)
type RecipeComponent = {
  type: "ingredient" | "semi_finished" | "operation"; // LOWERCASE
  componentId: string; // FK → ingredients.id / semi_finished_recipes.id
  componentName: string; // NOME SNAPSHOTATO – fallback se record eliminato
  quantity: number;
  unit: string; // "kg" | "unità" | "ore"
  pricePerUnit?: number; // Prezzo al momento della creazione
  costType?: string; // Per operations: "ENERGIA" | "LAVORO"
};

// ⚠️ Convenzione LEGACY (usata da foodMatrixEntries)
type FoodMatrixComponent = {
  type: "INGREDIENT" | "SEMI_FINISHED" | "FINAL_RECIPE" | "MANUAL"; // UPPERCASE
  sourceId: string;
  name: string;
  quantity: number;
  unit: string;
};
```

REGOLA DI NORMALIZZAZIONE: Il server ora normalizza automaticamente il tipo con `.toLowerCase()` prima di ogni lookup, garantendo retrocompatibilità.

5. Flusso di autenticazione



Flusso Google OAuth:

- Browser → GET /api/auth/google
 - Redirect a accounts.google.com (scope: openid email profile)
 - Callback GET /api/auth/google/callback?code=...
 - Scambio code per access_token (Google token endpoint)
 - GET userinfo (email, name, id)
 - Upsert user nel DB (crea se nuovo, aggiorna lastSignedIn)
 - Assegna store se utente nuovo
 - Crea JWT session cookie (1 anno)
 - Redirect a /

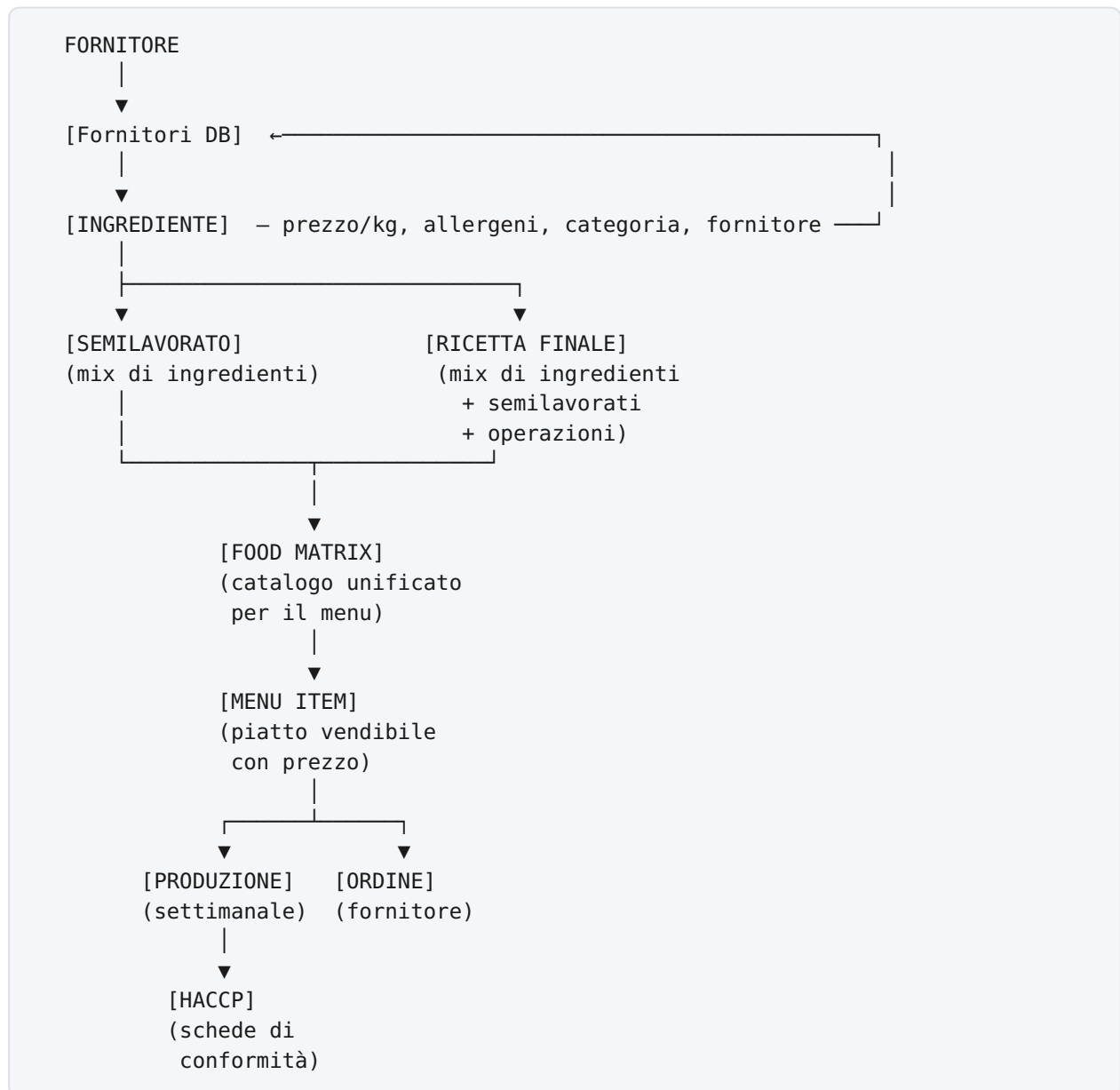
Flusso locale:

- POST /api/auth/login { email, password }
 - Lookup user by email
 - Verifica password (bcrypt hash)
 - Crea JWT session cookie
 - { success: true }

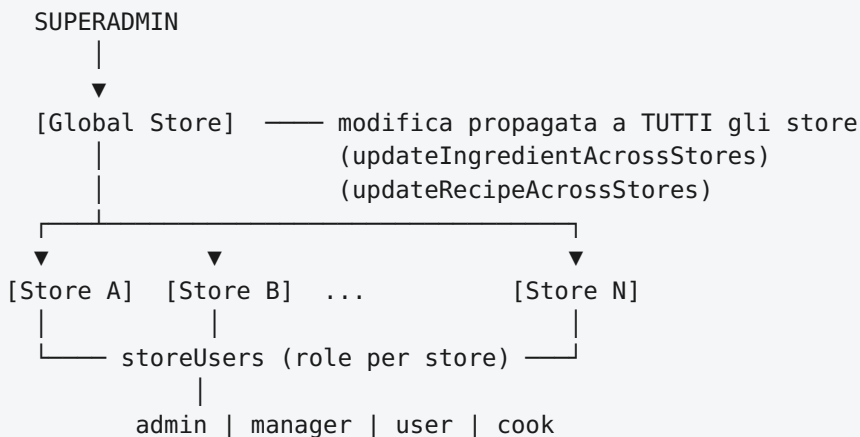
⚠ Apple Sign In: NON IMPLEMENTATO (richiede Apple Developer account)

⚠ Google Sign In: richiede GOOGLE_CLIENT_ID e GOOGLE_CLIENT_SECRET in .env

6. Flusso completo dati (dal fornitore al piatto)



7. Multi-store (multi-punto vendita)



8. Bug identificati e fix applicati

BUG 1: Ingredienti "Sconosciuto" nelle ricette (RISOLTO ✓)

File: `server/routers.ts` → procedure `finalRecipes.getDetails` **Causa:** Quando un ingrediente era eliminato o apparteneva a uno store diverso, il lookup `db.getIngredientById(comp.componentId)` restituiva `null` e il nome veniva impostato a `'Sconosciuto'` senza usare il campo `componentName` salvato nel JSON del componente.

```
// PRIMA (bug)
name: ingredient?.name || 'Sconosciuto'

// DOPO (fix)
name: ingredient?.name || comp.componentName || comp.name || 'Sconosciuto'
```

La stessa fix è stata applicata a `semi_finished` e `operation`.

BUG 2: Tipo componente case-insensitive (RISOLTO ✓)

File: `server/routers.ts`, `server/allergens.ts` **Causa:** Dati storici avevano `type: "INGREDIENT"` (uppercase) ma la logica confrontava con `'ingredient'` (lowercase), causando mancato riconoscimento dei componenti e quindi la lista ingredienti vuota nelle ricette.

Fix: Normalizzazione con `.toLowerCase()` prima di ogni confronto.

BUG 3: Google Sign In non visibile (CONFIGURAZIONE RICHIESTA)

File: `client/src/pages/LocalLogin.tsx` **Causa:** Il pulsante Google è implementato correttamente ma appare solo se il server conferma che `GOOGLE_CLIENT_ID` e `GOOGLE_CLIENT_SECRET` sono configurati. Senza queste variabili d'ambiente, il pulsante è nascosto.

Soluzione: Configurare nel file `.env` :

```
GOOGLE_CLIENT_ID=<ottieni da console.cloud.google.com>
GOOGLE_CLIENT_SECRET=<ottieni da console.cloud.google.com>
GOOGLE_REDIRECT_URI=https://tuo-dominio.com/api/auth/google/callback
```

BUG 4: Apple Sign In mancante (DA IMPLEMENTARE)

Apple Sign In non è mai stato implementato nel backend né nel frontend. Richiede:

1. Account Apple Developer con App ID e Service ID configurato
2. Chiave privata per firma JWT (ES256)
3. Endpoint `/api/auth/apple` e `/api/auth/apple/callback` nel backend
4. Pulsante nel frontend (simile a Google, condizionato da env var)

9. Schema Excel per importazione ingredienti

9.1 Intestazioni colonne (ordine fisso)

Colonna	Campo	Formato	Obbligatorio
A	Nome	Testo	☒
B	Categoria	Additivi / Alcolici / Bevande / Birra / Caffè / Carni / Farine / Latticini / Non Food / Packaging / Spezie / Verdura / Altro	☒
C	Tipo Unità	kg oppure unità	☒
D	Quantità Conf.	Numero (es. 5.000)	☒
E	Prezzo Conf. (€)	Numero (es. 12.50)	☒
F	Prezzo/kg o unità	Auto-calcolato = E/D (può essere lasciato vuoto)	☐
G	Fornitore	Testo (matchato fuzzy con DB suppliers)	☐
H	Tipo Confezione	Sacco / Busta / Brick / Cartone / Scatola / Bottiglia / Barattolo / Lattina / Sfuso	☐
I	Reparto	Cucina oppure Sala	☐
J	Marca	Testo	☐
K	Q.tà Min. Ordine	Numero	☐
L	Allergeni	Lista separata da virgola (es. "Glutine, Latte")	☐
M	È Alimento	SI oppure NO	☐
N	Ordinabile	SI oppure NO	☐
O	Note	Testo libero	☐

9.2 Esempio riga Excel

Nome		Cat.	U.	Q.Conf	P.Conf	P/kg	Fornitore	
Conf.	Rep.	Marca	QMin	Allergeni	Alim	Ord.	Note	

-- ----- ----- ----- ----- ----- ----- -----								
Farina 00		Farine	kg	25.000	18.50	0.74	Molino Rosso	
Sacco	Cucina	Le 5 St.	25	Glutine	SI	SI	Uso impasti	
Petto di pollo		Carni	kg	1.000	6.80	6.80	Macelleria P	
Sfuso	Cucina		0		SI	SI		
Olio EVO		Altro	kg	5.000	22.00	4.40	Oleificio B	
Bottiglia	Cucina	Zucchi	5		SI	SI	Extra Vergine	
Bicchieri 250ml		Packag	u	100.00	8.00	0.08	Office Depot	
Scatola	Sala		100		NO	SI	Usa e getta	

10. Logica di validazione e fuzzy matching all'importazione

Quando si importa un file Excel, il sistema esegue questi controlli:

IMPORT EXCEL



FASE 1: VALIDAZIONE STRUTTURA

- Controllo intestazioni colonne obbligatorie
- Conversione tipi (testo → numero, SI/NO → boolean)
- Validazione enum (categoria, tipo confezione, reparto)



FASE 2: FUZZY MATCHING FORNITORI

Per ogni riga con campo "Fornitore" compilato:

1. Cerca corrispondenza esatta nel DB suppliers
2. Se non trovato → applica distanza di Levenshtein
 - score > 0.85: match automatico (suggerito verde)
 - score 0.6–0.85: match probabile (suggerito giallo)
 - score < 0.6: nessuna corrispondenza (rosso → crea nuovo?)
3. Presenta all'utente le non-coincidenze con suggerimenti



FASE 3: MATCHING INGREDIENTI ESISTENTI

Per ogni riga importata:

- Cerca per nome esatto (case-insensitive)
- Se trovato → AGGIORNA prezzi (non crea duplicato)
- Se non trovato → CREA nuovo ingrediente



FASE 4: RIEPILOGO IMPORT

- N righe importate con successo
- N righe aggiornate
- N nuovi ingredienti creati
- Lista errori con descrizione
- Lista non-coincidenze fornitore per revisione manuale

11. Database correlati agli ingredienti (checks)

Database	Collegamento con Ingredienti	Status
suppliers	ingredients.supplierId (FK opzionale)	⚠️ FK non enforced nel DB, solo logica applicativa
semi_finished_recipes	components[].componentId → ingredient.id	✅ JSON ref
final_recipes	components[].componentId → ingredient.id	✅ JSON ref
food_matrix	sourceId → ingredient.id (se sourceType=INGREDIENT)	✅
food_matrix_entries	components[].sourceId → ingredient.id	✅ JSON ref
haccp	ingredients JSON field (snapshot)	✅ Snapshot
userOrderSessions	ingredientId FK con ON DELETE CASCADE	✅ Enforced
orderItems	itemId → ingredient.id (se itemType=INGREDIENT)	⚠️ No FK enforcement
wasteRecords	componentId → ingredient.id	⚠️ No FK enforcement

Nota: Le referenze nei campi JSON (components) non hanno FK enforcement a livello DB. Se un ingrediente viene eliminato, i componenti nelle ricette che lo referenziano mostreranno "Sconosciuto" (ora con fallback su componentName).

Raccomandazione: Prima di eliminare un ingrediente, verificare che non sia usato in ricette attive (aggiungere controllo server-side).

12. Struttura file principali

```
kitchen-management-app/
├── client/src/
│   ├── pages/
│   │   ├── LocalLogin.tsx      ← Auth UI (Google button condizionale)
│   │   ├── Ingredients.tsx     ← CRUD ingredienti + Excel I/O
│   │   ├── FinalRecipes.tsx    ← Ricette finali + dettaglio componenti
│   │   ├── FoodMatrix.tsx      ← Vista unificata ingredienti+ricette
│   │   └── ...
│   ├── components/
│   │   ├── RecipeForm.tsx      ← Form creazione/modifica ricette
│   │   └── ...
│   └── _core/hooks/useAuth.ts  ← Hook autenticazione
├── server/
│   ├── _core/index.ts          ← Entry point, registra tutte le route
│   ├── _core/googleAuthRoutes.ts ← Google OAuth
│   ├── _core/localAuthRoutes.ts ← Email/password auth
│   ├── routers.ts              ← Tutti i tRPC procedures (1561 righe)
│   ├── allergens.ts            ← Calcolo allergeni ricorsivo
│   ├── calculations.ts         ← Calcoli costo/resa
│   └── db.ts                   ← Layer database (tutte le query)
├── drizzle/
│   ├── schema.ts               ← Unica fonte di verità schema DB
│   └── 0000-0029_*.sql         ← Migration files
├── shared/
│   └── recipeValidation.ts      ← Validazione ricette condivisa client/server
└── ARCHITECTURE.md             ← Questo file
```

13. Prossimi passi raccomandati

Priorità Alta

1. **Completare logica fuzzy matching import** (vedi §10) con interfaccia UI di revisione
2. **Implementare Apple Sign In** (richiede Apple Developer Account)
3. **Aggiungere FK enforcement** per `orderItems.itemId` e `wasteRecords.componentId`
4. **Check pre-eliminazione ingredienti** — verificare utilizzo in ricette prima di cancellare

Priorità Media

1. **Ricette > Semilavorati** — verificare che `getSemiFinishedRecipes()` non ignori `storeId` dove necessario
2. **Food Matrix** — sincronizzare quando cambia il prezzo di un ingrediente

3. **HACCP ingredients snapshot** — aggiornare al cambio ricetta

Priorità Bassa

1. **Migrare tipi componenti uppercase** (FoodMatrix) a lowercase per uniformità
2. **Cache allergens** — il calcolo ricorsivo è costoso, considerare memoization